

Import Libraries

```
In [1]: #Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.express as px
import seaborn as sns
```

```
In [2]: #Load Data
df = pd.read_excel(r"C:\Users\user\Documents\Task 1\cleaned data.xlsx")
shape = df.shape
df.head()
```

Out[2]:

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	Paymer
0	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	I
1	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	
2	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	C
3	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	I
4	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	

Exploratory Data Analysis

```
In [3]: #Exploratory Data Analysis
print(df.shape)
df.info()
```

```
(1200, 14)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1200 entries, 0 to 1199
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   OrderID                1200 non-null   object
1   Date                   1200 non-null   datetime64[ns]
2   CustomerID             1200 non-null   object
3   Product                1200 non-null   object
4   Quantity               1200 non-null   int64
5   UnitPrice              1200 non-null   float64
6   ShippingAddress        1200 non-null   object
7   PaymentMethod          1200 non-null   object
8   OrderStatus            1200 non-null   object
9   TrackingNumber         1200 non-null   object
10  ItemsInCart            1200 non-null   int64
11  CouponCode             1200 non-null   object
12  ReferralSource         1200 non-null   object
13  TotalPrice             1200 non-null   float64
dtypes: datetime64[ns](1), float64(2), int64(2), object(9)
memory usage: 131.4+ KB
```

```
In [4]: df.describe()
```

Out[4]:

	Date	Quantity	UnitPrice	ItemsInCart	TotalPrice
count	1200	1200.000000	1200.000000	1200.000000	1200.000000
mean	2024-03-22 16:58:48	2.945833	356.412750	5.485000	1053.968300
min	2023-01-01 00:00:00	1.000000	11.390000	1.000000	11.390000
25%	2023-08-03 18:00:00	2.000000	186.062500	4.000000	410.520000
50%	2024-03-23 00:00:00	3.000000	364.210000	5.000000	823.615000
75%	2024-11-08 12:00:00	4.000000	521.570000	7.000000	1578.475000
max	2025-06-30 00:00:00	5.000000	699.930000	10.000000	3456.400000
std	NaN	1.407557	197.177146	2.281983	819.856558

```
In [5]: df.sort_values(by = "Date", inplace = True)
df.reset_index(drop=True, inplace=True)
df.head()
```

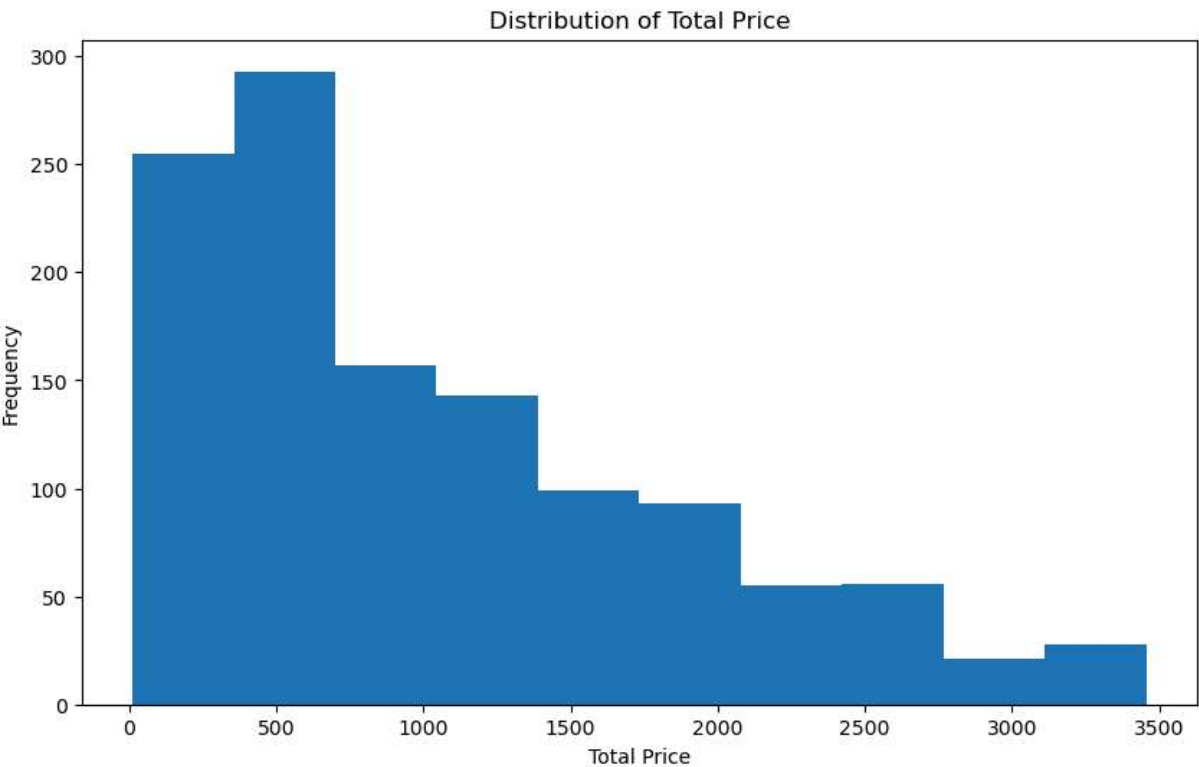
Out[5]:

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	Paymer
0	ORD200112	2023-01-01	C81366	Monitor	1	410.60	423 Main St	
1	ORD200848	2023-01-01	C54675	Chair	2	645.85	634 Main St	
2	ORD200236	2023-01-01	C14847	Monitor	1	318.81	562 Main St	I
3	ORD200373	2023-01-02	C60178	Laptop	3	341.15	969 Main St	
4	ORD200645	2023-01-02	C82990	Laptop	2	150.05	727 Main St	C

Univariate Analysis

In [6]:

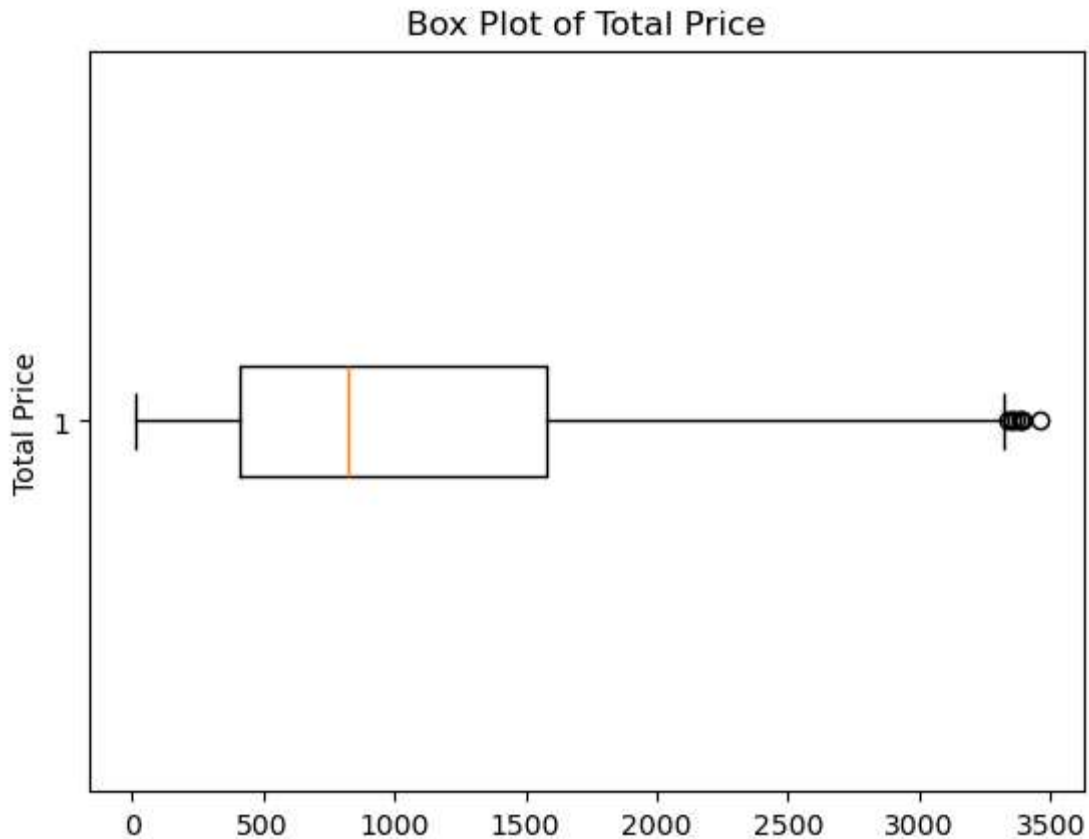
```
Fig, ax= plt.subplots(figsize=(10, 6))
df["TotalPrice"].hist(ax=ax)
ax.set_title("Distribution of Total Price")
ax.set_xlabel("Total Price")
ax.set_ylabel("Frequency")
plt.grid(False)
plt.show();
```



The distribution of total price is positively skewed, with a long tail extending toward higher transactions values. This indicates that most transactions occurs at lower to moderate prices,

while a relatively small number of high value transactions increased the spread of the data. This suggests that although expensive purchases are less frequent, they have a noticeable impact on the overall sales value.

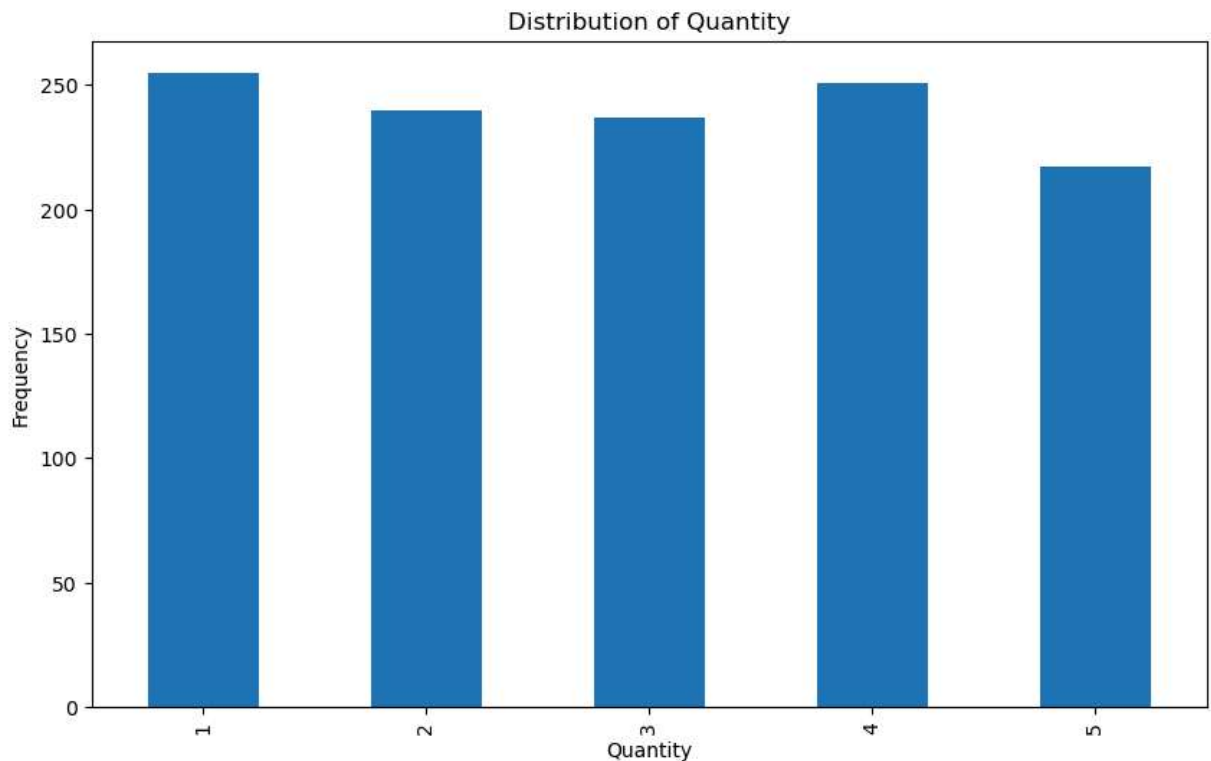
```
In [7]: plt.boxplot(df["TotalPrice"], vert=False)
plt.title("Box Plot of Total Price")
plt.ylabel("Total Price")
plt.show()
```



The boxplot indicates the presence of several high-value outliers in the total price distribution. Most transactions are concentrated within a relatively narrow price range, while a small number of unusually expensive transactions lie above the upper whisker. This supports the earlier observation that the distribution of total price is positively skewed.

```
In [8]: Fig, ax=plt.subplots(figsize=(10,6))
df["Quantity"].value_counts().sort_index().plot(kind="bar", ax=ax)
ax.set_title("Distribution of Quantity")
ax.set_xlabel("Quantity")
ax.set_ylabel("Frequency")

plt.show();
```



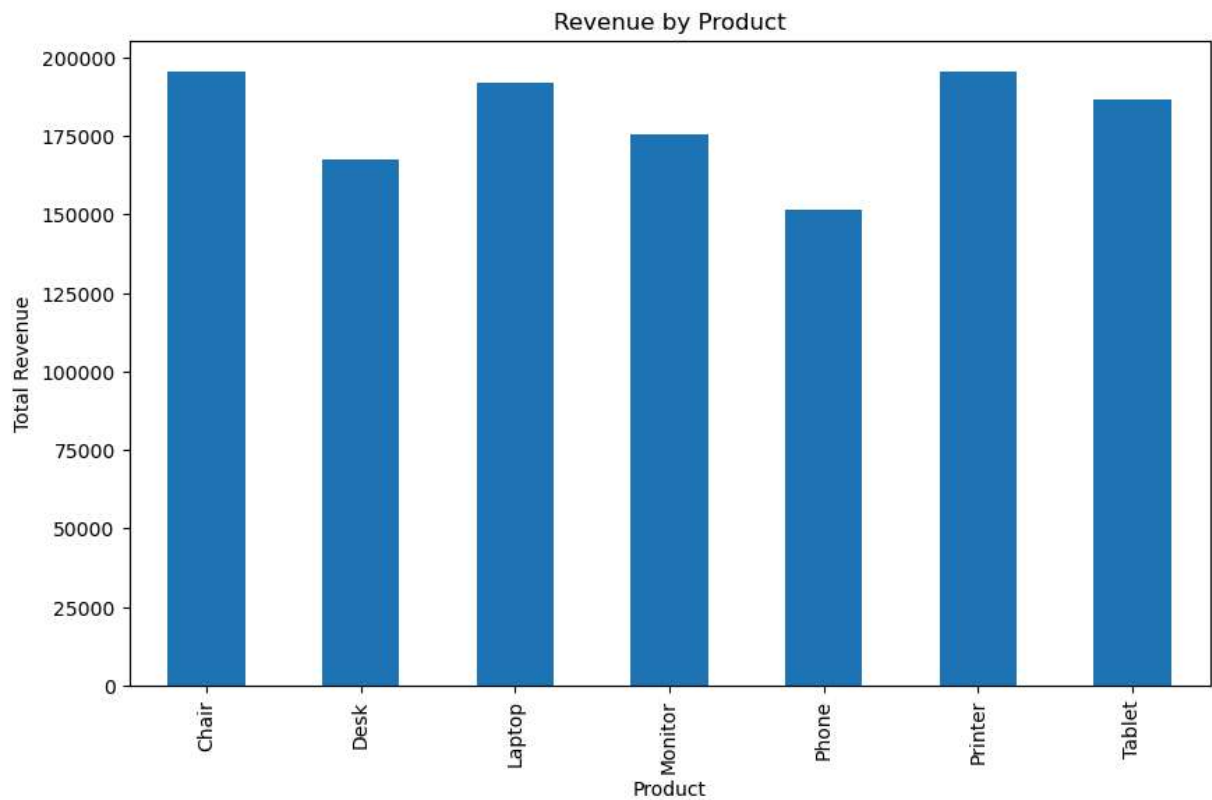
The distribution of product quantity purchased shows that most customers bought one product per transaction, making it the most common purchase quantity. This was followed by purchase of four products and five products were the least frequent. Overall, the distribution suggests that customers generally tend to purchase fewer items in a single transaction rather than buying in large quantities.

Bivariate Analysis

Which products generate the most revenue?

```
In [21]: product_revenue = df.groupby("Product")["TotalPrice"].sum()
Fig, ax = plt.subplots(figsize=(10, 6))
product_revenue.plot(kind="bar", ax=ax)
ax.set_title("Revenue by Product")
ax.set_ylabel("Total Revenue")
ax.set_xlabel("Product")

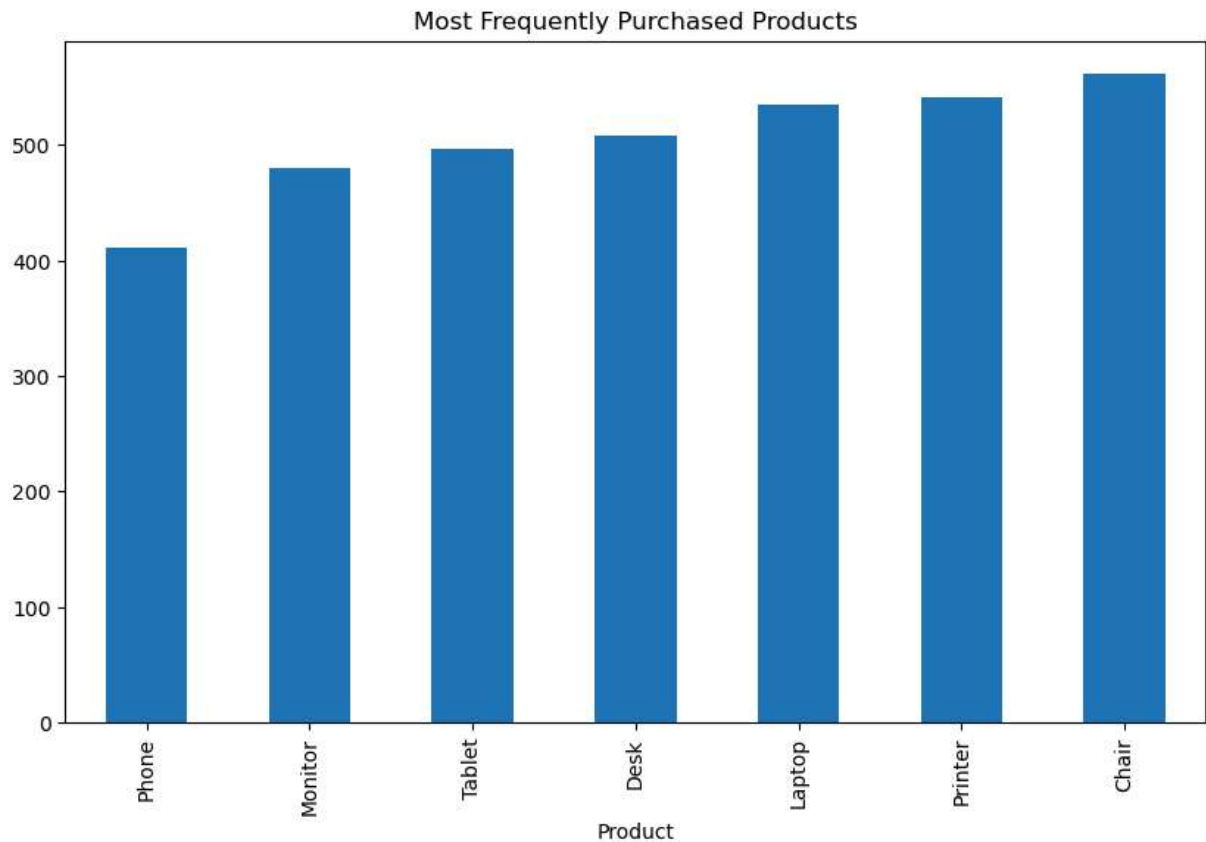
plt.show();
```



This analysis explores the product that generated the highest total revenue by summing the sales value of each product. It helps determine which product contributes the most to overall sales performance.

Which Product sold the highest number of units?

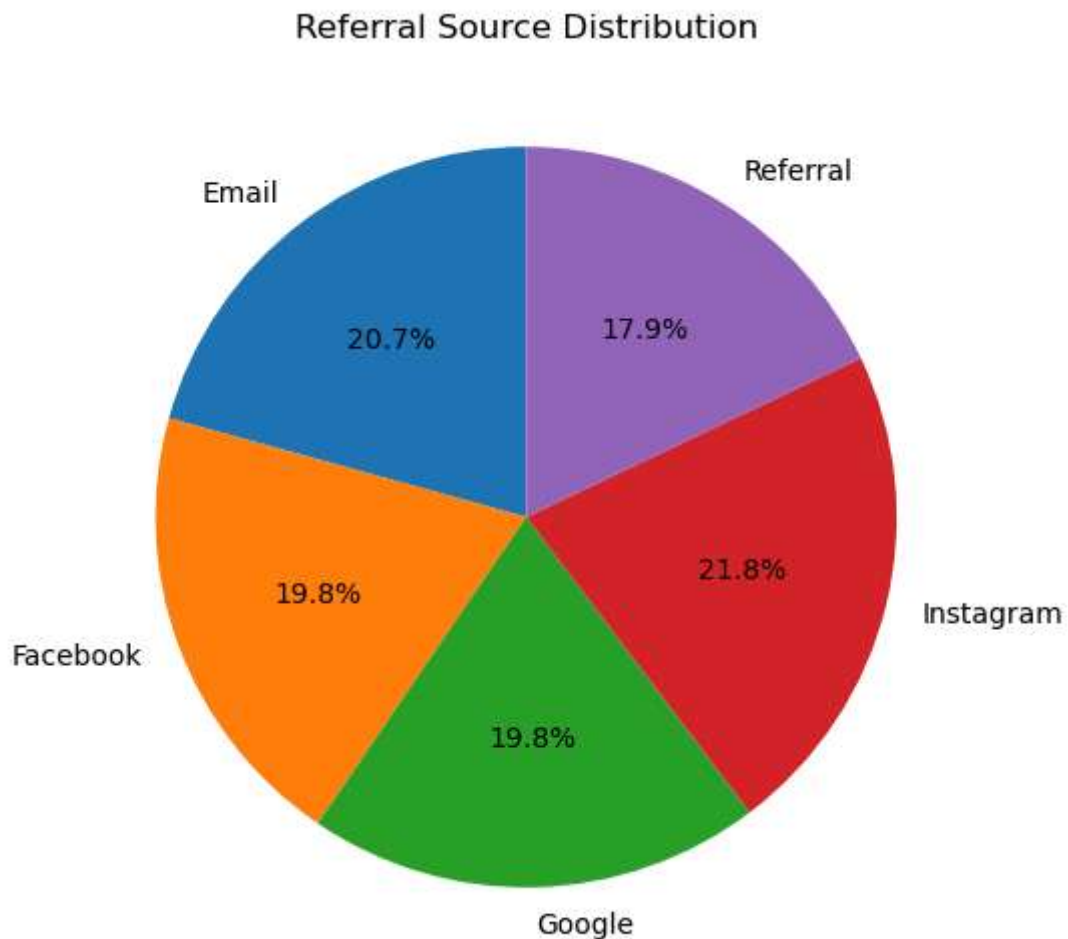
```
In [19]: product_quantity = df.groupby("Product")["Quantity"].sum().sort_values(ascending=True)
Fig, ax= plt.subplots(figsize=(10, 6))
product_quantity.plot(kind="bar", ax=ax, title= "Most Frequently Purchased Products")
plt.show()
```



This analysis identifies the product that customers purchased most frequently by counting the quantity of each product. Chair emerged as both the highest revenue generating product and the most frequently purchased product, highlighting its strong demand and contribution to sales. However, the rankings differ for some products. While Desk generated one of the lowest revenue, it ranked the fourth in purchase frequency. This indicates that although desk was purchased often, their lower selling price resulted in less overall revenue compared to higher priced products. Meanwhile phone remained the lowest revenue generating product, suggesting that revenue performance is influenced by both sales frequency and product price.

Which customer acquisition channel is the most preferred?

```
In [22]: Ref_TotalPrice= df.groupby("ReferralSource")["TotalPrice"].sum().reset_index()
Fig, ax = plt.subplots(figsize=(8, 6))
plt.pie(Ref_TotalPrice["TotalPrice"],
        labels= Ref_TotalPrice["ReferralSource"],
        startangle =90,
        autopct = "%1.1f%%"
        )
ax.set_title("Referral Source Distribution")
plt.show();
```

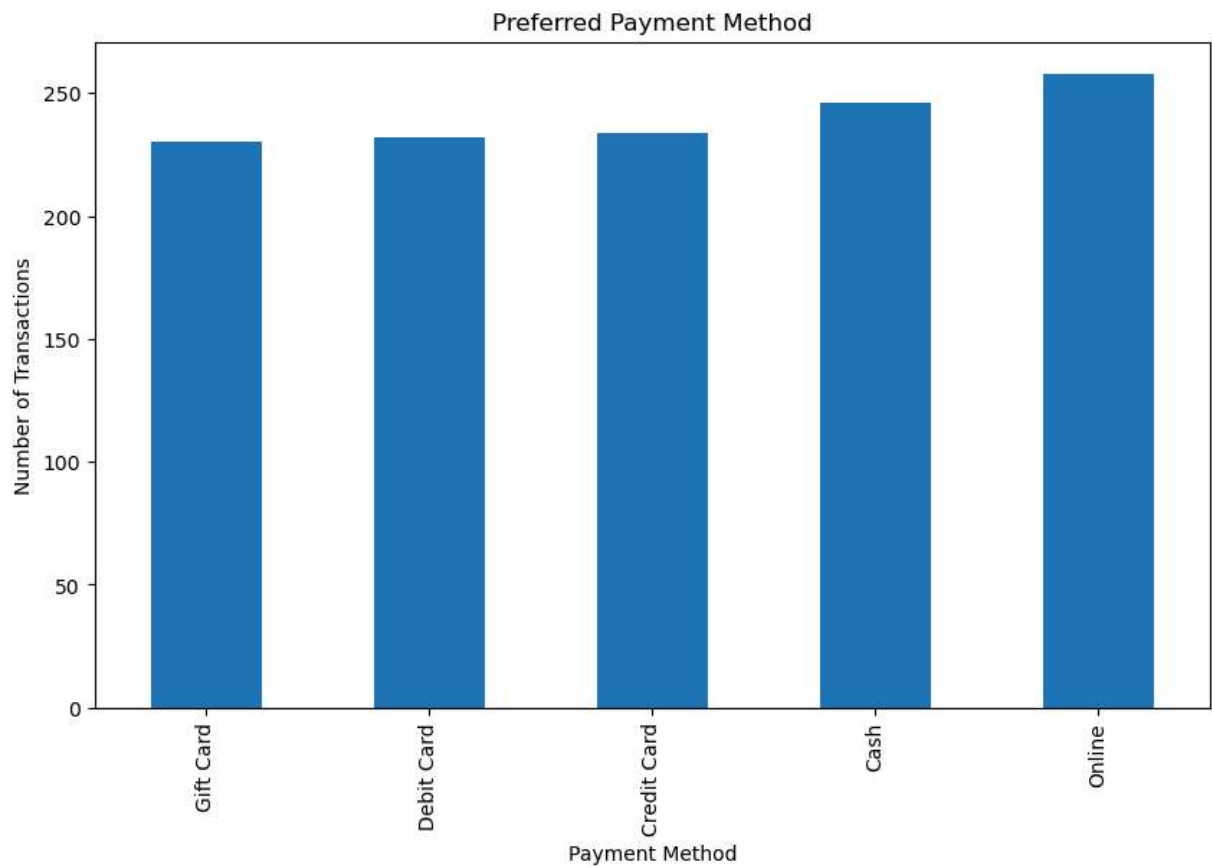


The Pie chart illustrates the proportion of total customers or sales contributed by each referral source. It highlights the most effective referral channel, showing that "Instagram" accounts for the largest share, while also revealing the relative contribution of the remaining sources.

Which payment methods do customers prefer?

```
In [12]: payment_counts= df["PaymentMethod"].value_counts().sort_values()
Fig, ax = plt.subplots(figsize=(10,6))
payment_counts.plot(kind="bar", ax=ax)
ax.set_title("Preferred Payment Method")
ax.set_xlabel("Payment Method")
ax.set_ylabel("Number of Transactions")

plt.show();
```

Online transaction was the most preferred payment method, recording the highest number of transactions, while Gift card is the least preferred payment method. This suggests that customers favoured this payment option over the other four options.

```
In [13]: cor_df = df.select_dtypes(include=["int64", "float64"])
sns.heatmap(cor_df.corr())
plt.show()
```



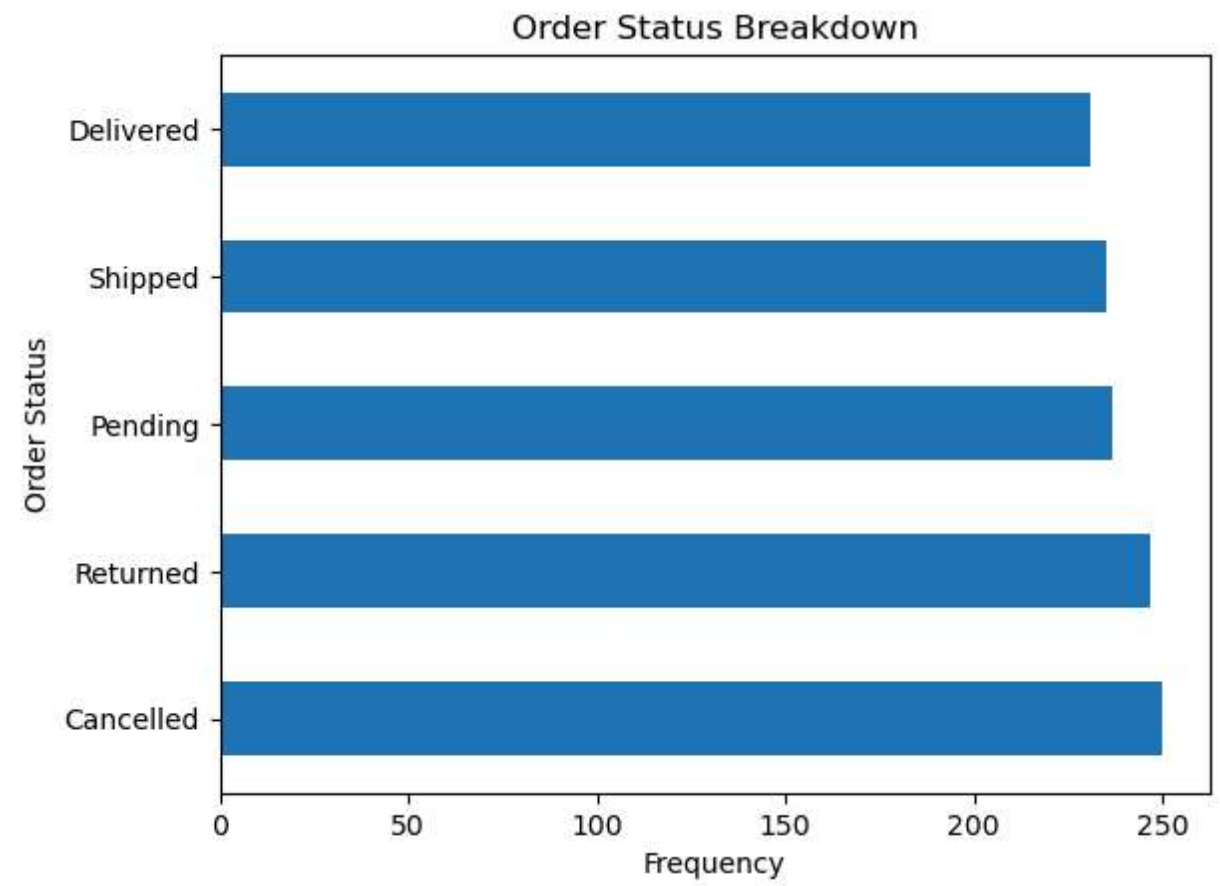
Correlation between columns with numerical values.

```
In [14]: monthly_sales = (
    df.set_index("Date")
    .resample("ME")["TotalPrice"]
    .sum()
)
moving_avg = monthly_sales.rolling(window=3).mean()
Fig, ax= plt.subplots(figsize=(15, 6))
monthly_sales.plot(ax=ax, color = "red", marker = 'o', label = "Monthly Sales")
moving_avg.plot(ax=ax, linewidth = 2, label = "3-Month Moving Average")
ax.set_title("Monthly Sales Across Different Years")
ax.set_xlabel("Date")
ax.set_ylabel("Total Price")
ax.legend()
plt.show()
```



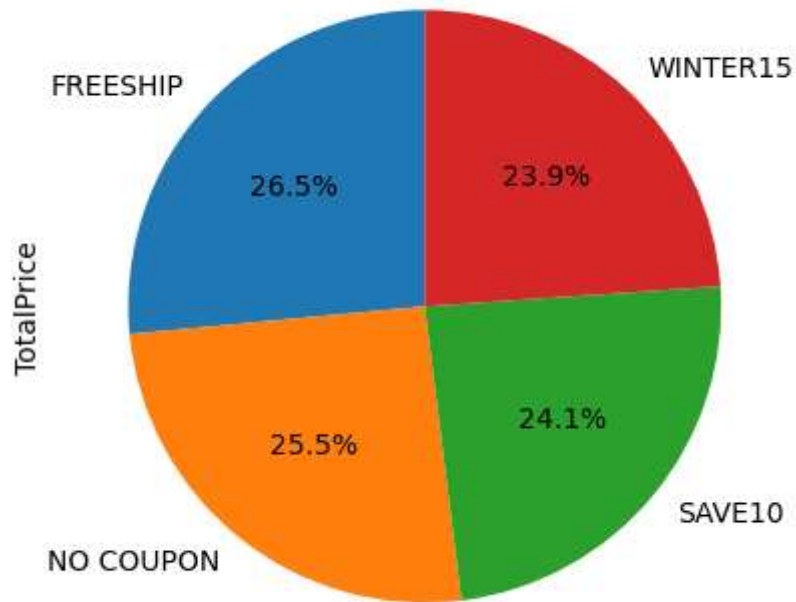
There is high variability across each months of respective year with occasional sharp peaks and declines. The 3-Month moving average reveals a smoother pattern, It indicates a gradual decline in sales from mid-2023 to early 2024, followed by a temporary recovery in mid-2024. Sales then remained relatively stable before showing signs of improvement towards the end.

```
In [15]: df["OrderStatus"].value_counts(sort=True).plot(kind="barh", title = "Order Status B
plt.ylabel("Order Status")
plt.xlabel("Frequency")
plt.show()
```



```
In [16]: Coupon_sales= df.groupby("CouponCode")["TotalPrice"].sum()  
Coupon_sales.plot(kind="pie", startangle =90, autopct ="%1.1f%%")
```

```
Out[16]: <Axes: ylabel='TotalPrice'>
```



```
In [ ]:
```